

Dealership Automation

Publish every car to several marketplaces from one stock list, keep them in sync, and follow up with buyers automatically

IN ONE SENTENCE

One stock list goes out to every marketplace and stays in sync; enquiries land in one inbox with polite automatic follow-ups.

BUILT WITH

Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	14

1. What this project is

Dealerships advertise each car on several marketplaces (AutoTrader-style portals, classified sites, their own website, social media). Staff re-type the same car into each site, forget to update prices, and leave sold cars online. Enquiries arrive in five different inboxes and some are never answered.

Dealership Automation keeps **one stock list** and does the repetitive work: every car is published to each **channel** in that site's own format, price and status changes go out automatically, sold cars come down everywhere, and **buyer enquiries from every site land in one inbox** with automatic follow-ups that are polite by design (quiet hours, weekly caps, STOP always works).

At a glance	
Type	Back-office web app with a background worker
Users	Dealer admin, sales manager, salespeople
Stack	Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker
Channels	Inventory feed (CSV/XML), signed webhook (any API, n8n, Zapier), two built-in sandbox marketplaces
Tests	13 shared unit tests, 18 API tests on a real database, full browser smoke test

Who uses it

User	What they do with it
Admin	Sets up channels and their secrets, dealership details, quiet hours, message limits and staff logins.
Manager	Manages stock and prices, switches cars on and off channels, writes follow-up rules, retries failed listings.
Salesperson	Sees stock and where each car is listed, answers buyers in the shared inbox.

2. The business problem

A typical used-car dealer lists 40–200 cars on three to six websites. Every price change is made by hand on each site, so prices disagree and sold cars stay up for days, which annoys buyers and wastes paid listing slots. Enquiries come by email, portal inbox and social messages; slow replies lose sales, and there is no record of who said what.

The client wants **one place to manage stock and enquiries**, with automation that is reliable (a site being down must not lose changes) and respectful (no spam, no messages at night, opt-outs honoured).

Pain point today	What it costs the business	How this project solves it
Re-typing each car into every site	Hours of admin per week; typos	Each car is published automatically in each site's format, checked against the site's rules
Price changes and sales not updated everywhere	Wrong prices shown; sold cars attract calls	Changes queue per listing and go out automatically; sold cars are removed everywhere
A site is down or rejects a listing	Changes silently lost	Retries with back-off (30 s → ~2 h), then a "Needs attention" list with one-click retry; every call logged
Enquiries spread across many inboxes	Slow or missed replies lose sales	One inbox for all channels; each enquiry stored once even if a site resends it
Follow-ups forgotten	Warm buyers go cold	Rules for instant reply, nudge after silence, price-drop notice and "sorry, it's sold"
Automation that spams	Complaints and blocked accounts	STOP opt-out, quiet hours, weekly cap per buyer, once per conversation, cancelled when the buyer replies

Value to the business

- Staff time moves from data entry to selling; prices are consistent everywhere within seconds.
- Every enquiry is answered fast, and the dealer keeps a complete conversation history per buyer and car.
- Works today with no one else's API keys: most portals accept an inventory feed, and a webhook reaches anything else.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Stock	Add and edit cars: stock number, VIN (validated), year, make, model, trim, mileage, fuel, gearbox, body, colour, price, description, https photo links, status (available, reserved, sold, withdrawn).
FR-02	Stock	Stock list with where each car is listed and the state on each channel (live, sending, can't list, failed, removed).
FR-03	Stock	Two people editing the same car get a conflict warning instead of overwriting each other.
FR-04	Channels	Channel types: inventory feed (CSV or XML at a secret address), webhook (signed JSON to any HTTPS endpoint), sandbox marketplaces for testing.
FR-05	Channels	Each channel has its own rules (title length, min/max photos, VIN required, whether "reserved" can be shown); a car that doesn't meet them is held back with the reason; fixable issues are fixed automatically and flagged.
FR-06	Channels	Switch each car on or off per channel; see each channel's activity log (every request and answer).
FR-07	Channels	Rotate a channel's secret; retry failed listings.
FR-08	Sync	Changes go through a queue: several quick edits while a site is down go out once, with the latest data; an unchanged save sends nothing (content hash).
FR-09	Sync	Failed calls retry after 30 s, 2 min, 8 min, 32 min and ~2 h, then wait under "Needs attention". A car already live stays up with its last good version.
FR-10	Inbox	Receive enquiries through a signed endpoint per channel (or the sandboxes' "Simulate an enquiry"); each stored once however often it is resent.
FR-11	Inbox	One conversation per buyer, car and channel; staff reply from the inbox; replies go through the same queue so a down channel doesn't lose them.
FR-12	Inbox	A buyer who replies STOP (or similar wording) gets nothing more from people or automation.
FR-13	Follow-ups	Rules with triggers: new enquiry, no reply after N minutes, price drop, sold. Templates with fields such as {buyer_first_name}, {vehicle}, {price}, {old_price}, {link}, with a live preview.
FR-14	Follow-ups	Guards always apply: quiet hours (message waits), at most N automatic messages per buyer per week, once per rule per conversation, cancelled if the car has gone or the buyer replied.
FR-15	Admin	Dashboard: stock on channels, failures needing attention, unanswered conversations, messages sent.
FR-16	Admin	Settings (dealer name, phone, website, currency, quiet hours, weekly cap), staff logins with roles, activity log.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Security	Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.
NFR-02	Security	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-03	Security	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-04	Security	An activity (audit) log of logins and every business action, viewable by admins.
NFR-05	Security	Webhooks we send carry <code>x-timestamp</code> and an HMAC-SHA256 <code>x-signature</code> ; enquiries we receive must be signed the same way and be under 5 minutes old (replay protection). Signatures and feed secrets are compared in constant time.
NFR-06	Reliability	Jobs are claimed with <code>FOR UPDATE SKIP LOCKED</code> , so several workers never take the same job; channel calls happen outside database transactions; a job stuck "running" for 10 minutes is picked up again; creating a listing twice is harmless.
NFR-07	Reliability	The database enforces one waiting job per listing, one car per VIN in stock, one follow-up per rule per conversation, and each inbound message stored once.
NFR-08	Time	Quiet hours follow the dealership's time zone, correctly across midnight and daylight-saving changes.
NFR-09	Operations	Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances.
NFR-10	Operations	Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.
NFR-11	Operations	The worker runs inside the web process by default, or as a separate process (<code>WORKER=off</code> on web servers).

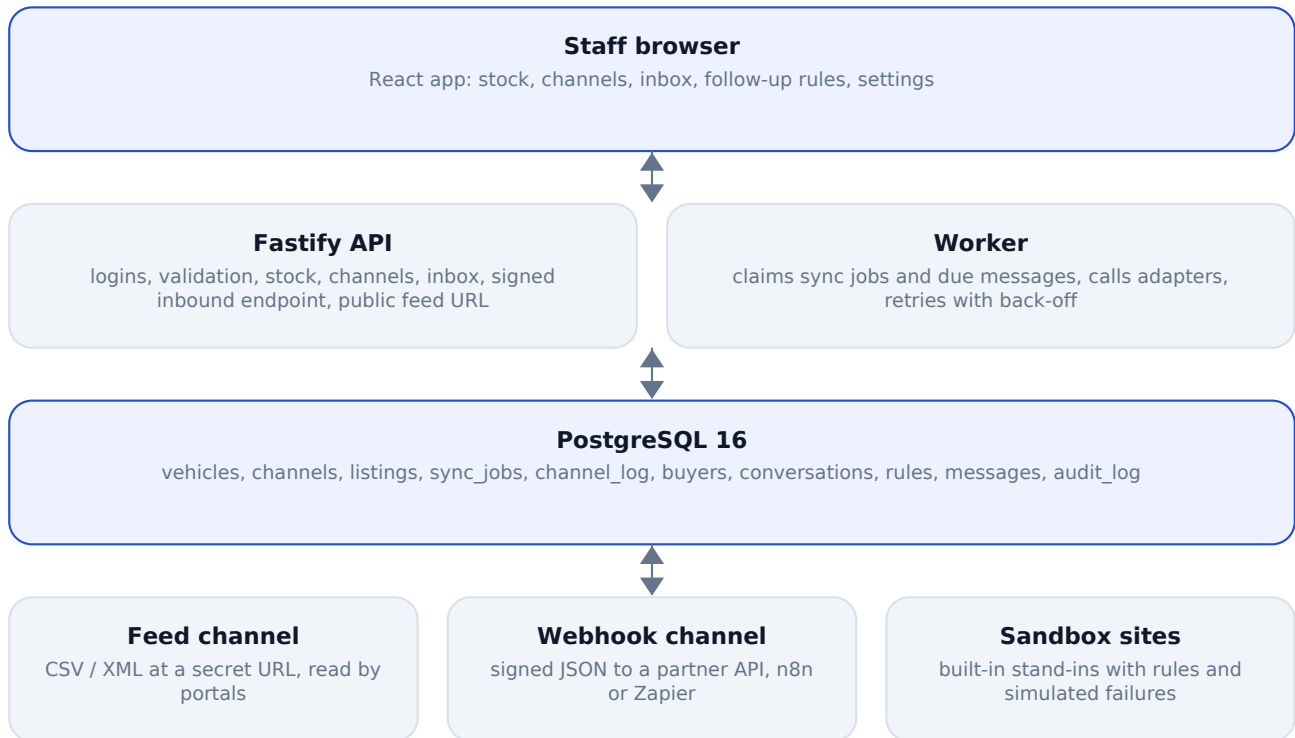
Out of scope for this release

- Direct adapters for specific portals' private APIs (AutoTrader Connect, eBay Motors, Facebook partner APIs) — they need accounts only the dealer can obtain; they slot in beside the existing adapters.
- SMS/WhatsApp sending (messages go through the channel that the enquiry came from).

4. How the solution is designed

The web app and the background worker share one codebase and one database. Staff actions only change the database and enqueue work; the worker does the slow, failure-prone part (talking to other sites) with retries, so the web app stays fast and nothing is lost when a site is down.

Architecture



Technology choices

Part	Choice	Why
Server	Node 22 + Fastify 5	Small and fast, TypeScript throughout; the worker is plain code in the same process or its own.
Database	PostgreSQL 16	The job queue lives in the database (<code>SKIP LOCKED</code>), so there is no extra queue service to run.
Shared rules	<code>shared/automation.ts</code>	Channel rules, rendering, templates, opt-out detection, quiet hours and retry delays as pure, unit-tested functions.
Web app	React 19, TanStack Query, React Router	Live-updating lists (polling), forms with server validation messages.
Tests	Vitest, Playwright	API tests on a real database with fake adapters; browser run of a real day's work.
Delivery	Docker, GitHub Actions	One image; CI runs every test and boots the image.

Data model

Table / file	What it holds
vehicles	Stock with price, status, photos and a version number for conflict detection.

Table / file	What it holds
channels	Name, type (sandbox, webhook, feed), settings and the secret used for signatures and the feed URL.
listings	One per car and channel: wanted or not, state, the site's id, the hash of what was last published, problems.
sync_jobs	"Bring this listing in line with the car": status, attempts, run-at time; one waiting job per listing, with a re-run flag.
channel_log	Every request and answer per channel with status and duration.
buyers / conversations / messages	Buyers (with opt-out), one conversation per channel thread, inbound and outbound messages with scheduling and status.
rules	Follow-up rules: trigger, delay, template, enabled.
settings / users / sessions / audit_log	Dealer settings, staff, sessions, activity log.

Key business rules

- **Plan, then send:** when a car changes, the system decides per channel what the listing should be (create, update, remove or blocked by rules) and enqueues one job. The worker renders the listing from the car **as it is when the job runs**, so three quick edits send once.
- **Nothing unchanged is sent:** a listing whose rendered content hash equals the last published hash is skipped.
- **Retries:** 30 s, 2 min, 8 min, 32 min, ~2 h (6 attempts), then "dead" until a person retries.
- **Automatic message guards** are checked when the message is about to be sent, not only when it is scheduled: opted out, quiet hours, weekly cap, car gone, buyer replied.

Folder layout

```

shared/  schemas.ts · automation.ts (rules, rendering, templates, quiet hours) · types.ts
server/  src/sync/adapters.ts · engine.ts · messages.ts · src/worker.ts
         src/routes/vehicles.ts · channels.ts · inbox.ts · admin.ts · auth.ts
         migrations/001_init.sql · test/automation.test.ts
web/     src/pages (Dashboard, Stock, CarPage, Channels, Inbox, Rules, Settings, AuditLog)
         scripts/smoke.mjs
docs/    OPERATIONS.md · PROJECT-GUIDE.pdf

```

5. Build it from scratch, step by step

Build the foundation first, then stock, then publishing (the hardest part), then the inbox and follow-ups. Keep channel-specific logic behind a small adapter interface so new sites can be added later.

Before you start

- Node.js 22+, PostgreSQL 16, Git.
- Comfort with TypeScript, SQL and React; understanding of HMAC signatures helps.

Step 1. Set up the workspace

One repository with a server, a web app and shared code used by both.

- 1.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir dealership-automation && cd dealership-automation && git init
npm init -y # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg zod
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

- 1.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 2. Configuration, database connection and migrations

- 2.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`, `WORKER`, `WORKER_INTERVAL_MS`) and refuse to start with a clear message when one is wrong.
- 2.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.
- 2.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 3. Design the database schema

- 3.1 Create the tables in the data model above. Put the rules in the database: a unique index for one car per VIN while in stock, a partial unique index for one waiting job per listing, one follow-up per rule and conversation, and each inbound message stored once.

```
CREATE UNIQUE INDEX sync_jobs_one_waiting ON sync_jobs (listing_id) WHERE status IN ('queued', 'running');
CREATE UNIQUE INDEX messages_rule_once ON messages (conversation_id, rule_id, rule_key) WHERE rule_id IS NOT NULL;
CREATE UNIQUE INDEX messages_inbound_once ON messages (conversation_id, external_id) WHERE direction = 'in';
```

Done when: The schema migrates on a fresh database.

Step 4. Logins, roles and request security

The same proven foundation every production app here uses.

- 4.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 4.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 4.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 4.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 4.5 **Roles:** admin (channels, settings, users), manager (stock, follow-ups, retries) and sales (stock view, inbox). A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 4.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 4.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 5. Shared automation rules

- 5.1 In `shared/automation.ts` write pure functions: `checkVehicle(car, channelRules)` (errors and warnings), `renderListing(car, channel)` (title shortening on a word boundary, photo limits), `renderTemplate(text, fields)`, `isOptOut(text)` (STOP, unsubscribe...), `nextSendTime(now, quietStart, quietEnd, timeZone)` and `retryDelayMs(attempt)`.
- 5.2 Unit-test the edge cases: quiet hours across midnight and on the day the clocks change, templates with unknown fields refused.

Done when: Shared unit tests pass.

Step 6. Stock management

- 6.1 API: list, create, update (with `version` for conflicts), change status. Every save calls the planner for each channel.
- 6.2 Web: stock list with a status dot per channel, car page with the form and a per-channel panel showing problems and a switch.

Done when: Editing a car updates its row and shows "Sending" on each channel it is listed on.

Step 7. Channels and adapters

- 7.1 Define an adapter interface `publish / update / remove / sendMessage` returning `{ ok, status, externalId, detail }`.
- 7.2 **Sandbox adapter:** stores what a marketplace would show in `sandbox_listings`, with configurable failure rate for testing; deterministic ids.
- 7.3 **Webhook adapter:** POSTs JSON events (`listing.created`, `listing.updated`, `listing.removed`, `message.send`) with `x-timestamp` and `x-signature = HMAC-SHA256(secret, timestamp.body)`; 5xx and timeouts are retried, 4xx are not.
- 7.4 **Feed adapter:** nothing to send; `GET /feeds/:channelId/:secret.csv|xml` renders the live listings (CSV protected against formula injection, XML escaped).
- 7.5 Channels page: create, edit, rotate secret, activity log, retry failed.

Done when: Each adapter type has API tests (signature verified, 5xx retried, 4xx not, feed content correct).

Step 8. The sync engine and worker

- 8.1 `enqueue(listing)`: insert a queued job, or set `rerun` on the waiting one (the unique index makes this safe).
- 8.2 `runJobs()`: claim due jobs with `FOR UPDATE SKIP LOCKED`, mark them running, **commit**, then call the adapter outside the transaction, then record the result, log it in `channel_log`, and schedule a retry or mark done/dead.
- 8.3 Skip unchanged listings by comparing the content hash; re-queue jobs stuck in "running" for 10 minutes.
- 8.4 Start the worker on an interval inside the server (`WORKER=inline`) or as `node dist/cli/worker.js`.

Done when: Tests: three quick price edits while a channel fails produce one successful update with the last price after retries.

Step 9. Inbox and inbound enquiries

- 9.1 `POST /api/inbound/:channelId`: verify the HMAC over the **raw request body** and the timestamp (reject if older than 5 minutes), validate, find or create the buyer and conversation, store the message once (unique external id).
- 9.2 Detect opt-out wording and mark the buyer opted out; a buyer reply cancels pending nudges.
- 9.3 Inbox page: conversation list with unread markers, thread view, reply box (replies are queued messages sent by the worker).

Done when: Tests: a replayed or unsigned enquiry is refused; the same message twice is stored once; STOP stops everything.

Step 10. Follow-up rules

- 10.1 Rules page with a template editor and live preview; triggers new enquiry, no reply after N minutes, price drop and sold.
- 10.2 Schedule messages when events happen (new enquiry, price lowered, car sold), and scan for silent conversations for the "no reply" rule.
- 10.3 `sendDue()` re-checks every guard at send time: opt-out, quiet hours (reschedule), weekly cap, car gone, buyer replied, and records why a message was held back.

Done when: Tests cover each guard, including quiet hours and the weekly cap.

Step 11. Web app shell

- 11.1 `api/client.ts`: a small `fetch` wrapper that sends the CSRF token on writes, turns error responses into readable messages (with per-field details for forms), and sends the user to sign-in on 401.
- 11.2 `api/auth.ts`: TanStack Query hooks for the current user, sign-in and sign-out; a layout with navigation that shows only what the role may use.
- 11.3 Styles with light and dark mode and a phone layout (tables scroll inside their card, forms stack to one column).

Done when: You can sign in, see the navigation for your role and sign out.

Step 12. Dashboard, settings, activity log and demo data

- 12.1 Dashboard with counts and a "Needs attention" list (failed listings, cars that can't be listed, unanswered buyers).
- 12.2 Settings and user management for admins; activity log with readable labels.
- 12.3 `seed.ts`: stock with photos, two sandbox marketplaces with different rules, a feed channel, follow-up rules and some buyer conversations (guarded in production by `ALLOW_DEMO_SEED`).

Done when: `npm run seed:demo && npm run dev` shows a lively demo.

Step 13. Test it end to end

- 13.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 13.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 13.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then drive a day's work: a price change reaching every channel, a channel switched off, a new car published, an enquiry answered automatically, a manual reply, a sale, an opted-out buyer, the feed, and phone layouts. Fail on any script error, and save screenshots.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 14. Package, automate and document

- 14.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 14.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 14.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 14.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **13 shared unit tests:** channel rules, rendering, templates, opt-out wording, quiet hours across midnight and a clock change.
- **18 API tests on PostgreSQL:** publishing rules, coalescing, retries and dead-lettering, signatures and replays, duplicates, STOP, price drops, sales, nudges, quiet hours and weekly limits, the feed, webhooks (5xx retried, 4xx not), permissions and per-session rate limits.
- **Browser smoke test** of a full day in Chromium, including phone layouts.
- Bugs found and fixed during the build included an SQL parameter needing a `::timestampz` cast (it would have broken the "no reply" nudges) and a retry path that violated the one-waiting-job rule.

```
npm run typecheck
npm test
npm run build && npm run smoke
```

Deploying and running it

```
cp .env.example .env # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email you@dealer.com --name "Your Name"
# demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js
# logins admin@ / manager@ / sales@demo.local, password demo-password-1
```

- To connect a real portal: if it takes a feed, add a feed channel and send them its address; if it has an API, point a webhook channel at a small adapter or an n8n/Zapier flow.

Known limits

- No direct adapters for private portal APIs yet (feed and webhook cover most cases).
- Messages to buyers go back through the channel; there is no SMS/WhatsApp sender.
- Rate limits are counted per server instance.

Sensible next steps

- Direct adapters for the dealer's main portals once API keys are available.
- Take stock automatically from the dealer management system instead of typing it.
- Send marketplace enquiries into the enquiry and sales dashboard for reporting.